Task 3.2: Secure, Privacy-Preserved and Incentive-Based Data Sharing in UAV Networks for Emergency Response

COVER Project: EU MSCA Staff Exchanges, Grant No. 101086228

Henry Kabuye

Northumbria University

Supervisor: Professor Nauman Aslam

COVER Project: Task 3.2 (Lead: Georg-August-Universität Göttingen)

Supported by: CentraleSupelec (CS) and UESTC

May 2026

Abstract

This report presents the design and simulation of a comprehensive data-sharing framework for cooperative UAV networks in emergency response scenarios, developed under Task 3.2 of the COVER project. The framework integrates five tightly coupled components: (i) a blockchain-based security layer using Proof-of-Authority (PoA) consensus to defend against Sybil, Byzantine, and free-rider attacks; (ii) a Stackelberg game-theoretic incentive mechanism with Proof-of-Resource-Sharing (PoRS) that elicits truthful participation from heterogeneous UAV agents; (iii) Differentially Private Stochastic Gradient Descent (DP-SGD) with tight Rényi Differential Privacy (RDP) accounting for gradient privacy; (iv) Federated Learning (FL) with convergence analysis under 3D UAV mobility and bandwidth-limited wireless channels; and (v) a *Multi-Agent Reinforcement Learning (MARL) agentic framework* in which each UAV independently learns an adaptive participation policy via Deep Q-Networks (DQN), with Q-network weights periodically federated across agents. All components are implemented in a Python simulation framework with round-robin multi-GPU support; results in this report were run on a single NVIDIA GB10 (Grace Blackwell, DGX Spark-class, unified memory), with DP-SGD accelerated via vectorised per-sample gradients (`torch.func.vmap`). Evaluation uses CIFAR-10 as a standard FL benchmark, AIDER (real UAV disaster imagery) as an on-task proxy, and EuroSAT (satellite/aerial land-cover) as a third, more class-balanced proxy (Section 8.10). The central empirical finding, reached only after tracing and correcting three layered privacy-accounting bugs (Section 8), is that a single noise level calibrated to a heterogeneous UAV fleet's smallest client is the wrong way to add differential privacy to small-cohort federated learning: at a genuinely valid per-client guarantee, standard uniform-noise DP-SGD barely learns, while a new per-client adaptive noise calibration mechanism, holding an *identical* formal guarantee, recovers substantial learning on CIFAR-10 (+13.4 percentage

points, 5-seed mean) and EuroSAT (+16.3pp, 3-seed mean), though it does not rescue every dataset tested (Section 8.2).

Contents

1 Introduction **3**

2 System Model **3**
2.1 UAV Network Topology . . . 3
2.2 Wireless Channel Model . . . 3

3 Federated Learning with Differential Privacy (Sub-tasks C & D) **3**
3.1 Federated Averaging . . . 3
3.2 DP-SGD Local Training . . . 4
3.3 Privacy Accounting . . . 4

4 Incentive Mechanism (Sub-task B) **4**
4.1 Stackelberg Game Formulation . . . 4
4.2 Proof-of-Resource-Sharing (PoRS) . . . 5

5 Blockchain Security Layer (Sub-task A) **5**
5.1 Proof-of-Authority Consensus . . . 5
5.2 Attack Models and Defences . . . 6

6 Multi-Agent Reinforcement Learning Agentic Framework **6**
6.1 Motivation and Problem Statement . . . 6
6.2 Markov Decision Process Formulation . . . 6
6.3 Deep Q-Network Architecture . . . 7
6.4 Federated Q-Learning . . . 8
6.5 Integration with the Full Stack . . . 9

7 Implementation **9**

8 Results **10**
8.1 Convergence Under Differential Privacy . . . 12
8.2 Per-Client Adaptive Noise Calibration . . . 13
8.3 Incentive Mechanism Convergence . . . 15
8.4 Blockchain Performance . . . 15
8.5 MARL Agentic Framework Results . . . 16
8.6 Comparison to Published DP-FL Results on CIFAR-10 . . . 18
8.7 MobileNetV2 Validation: A Negative Result . . . 20
8.8 Real UAV Imagery (AIDER): Integration and a Convergence Diagnosis . . . 21
8.9 Adaptive Clipping: Re-Validated, and It Breaks Training at the Corrected σ 24
8.10 EuroSAT: From a Non-Sustained Signal to Real Learning . . . 25

9 Conclusion **27**

1 Introduction

The COVER project (Cooperative and Intelligent Unmanned Aerial Vehicles for Emergency Response Applications) aims to develop intelligent, connected UAV networks capable of supporting first responders in extreme-weather events such as flooding, wildfires, and earthquakes. Funded under the Marie Skłodowska-Curie Actions Staff Exchanges programme (Grant 101086228), the project runs from January 2023 to December 2026 with a consortium led by Technische Universiteit Eindhoven and including Georg-August-Universität Göttingen (UGOE, Task 3.2 lead), CentraleSupelec, KTH, and twelve associated institutions.

Task 3.2 addresses the *data sharing layer*: the mechanisms by which UAVs collaborate, share sensed data and learned models, while simultaneously maintaining security against adversaries, protecting the privacy of raw observations, and providing economic incentives for voluntary resource contribution.

This document describes the methodology, system model, and simulation framework developed by the author at Northumbria University in the context of Task 3.2, led by UGOE within the COVER consortium.

2 System Model

2.1 UAV Network Topology

We consider $N = 20$ UAVs deployed in a $1000 \times 1000 \times [50, 200]$ m three-dimensional operational area. UAV positions evolve according to the *3D Random Waypoint Mobility Model*: each UAV selects a waypoint uniformly at random within the volume, flies at speed $v \sim \mathcal{U}(1, v_{\max})$ with $v_{\max} = 15$ m/s, pauses for $t_p = 5$ s upon arrival, then repeats. UAV 0 acts as the parameter server (PS); UAVs $1, \ldots, N-1$ are worker nodes.

2.2 Wireless Channel Model

Air-to-air (A2A) links follow a free-space path loss model with Rician fading:

$$\mathrm{SNR}_{ij} = P_{\mathrm{tx}} - \mathrm{FSPL}(d_{ij}) + G_{\mathrm{Rician}}, \tag{1}$$

where $\mathrm{FSPL}(d) = 20 \log_{10}\left(\frac{4\pi d f}{c}\right)$, $f = 5.8$ GHz, and the Rician power gain G_{Rician} has K-factor $K = 10$ dB, capturing the dominant line-of-sight component typical in open-sky UAV operations [1]. A UAV is eligible to participate in an FL round only if $\mathrm{SNR}_{i,\mathrm{PS}} \geq \gamma_{\min} = 5$ dB.

3 Federated Learning with Differential Privacy (Subtasks C & D)

3.1 Federated Averaging

Global model parameters $\mathbf{w}$ are updated via FedAvg [2]:

$$\mathbf{w}^{(t+1)} = \sum_{k \in \mathcal{S}^{(t)}} \frac{n_k}{N_{\mathcal{S}}} \mathbf{w}_k^{(t)}, \tag{2}$$

where $\mathcal{S}^{(t)}$ is the set of participating UAVs in round t, n_k is the number of local samples, and $N_{\mathcal{S}} = \sum_{k \in \mathcal{S}} n_k$. Each selected UAV runs $E = 5$ local SGD epochs on its private dataset before uploading the updated weights.

Data heterogeneity across UAVs is modelled via a Dirichlet distribution: class-conditional indices are split according to $\mathbf{p}_c \sim \mathrm{Dir}(\alpha \mathbf{1}_N)$ with $\alpha = 0.5$, producing the non-IID local datasets that are standard in FL research.

3.2 DP-SGD Local Training

To prevent the PS from inferring sensitive information from individual model updates, each participating UAV applies DP-SGD [3]:

Algorithm 1 DP-SGD at UAV k, round t

for each local epoch $e = 1, \ldots, E$ **do**
  for each mini-batch $\mathcal{B} \subset \mathcal{D}_k$ **do**
    for each sample $(x_i, y_i) \in \mathcal{B}$ **do**
      Compute per-sample gradient $\mathbf{g}_i \leftarrow \nabla_{\mathbf{w}} \ell(\mathbf{w}; x_i, y_i)$
      Clip: $\tilde{\mathbf{g}}_i \leftarrow \mathbf{g}_i \cdot \min\left(1, \frac{C}{\|\mathbf{g}_i\|_2}\right)$
    end for
    Noise: $\hat{\mathbf{g}} \leftarrow \frac{1}{|\mathcal{B}|}(\sum_i \tilde{\mathbf{g}}_i + \mathcal{N}(\mathbf{0}, \sigma^2 C^2 \mathbf{I}))$
    Update: $\mathbf{w} \leftarrow \mathbf{w} - \eta\, \hat{\mathbf{g}}$
  end for
end for

The clipping threshold is $C = 1.0$ and noise multiplier $\sigma = 1.1$.

3.3 Privacy Accounting

Privacy expenditure is tracked with the Rényi Differential Privacy (RDP) accountant [4,5]. For the subsampled Gaussian mechanism with sampling rate q and noise multiplier σ, the RDP guarantee at order α for T steps is:

$$\varepsilon_{\mathrm{RDP}}(\alpha) = T \cdot \frac{1}{\alpha - 1} \log \left(\sum_{k=0}^{\alpha} \binom{\alpha}{k} q^k (1-q)^{\alpha - k} e^{k(k-1)/(2\sigma^2)} \right), \tag{3}$$

which is then converted to (ε, δ)-DP via [6]:

$$\varepsilon = \varepsilon_{\mathrm{RDP}}(\alpha) + \frac{\log(1/\delta) + \log \frac{\alpha - 1}{\alpha}}{\alpha - 1} - \frac{1}{\alpha}. \tag{4}$$

Training stops automatically when $\varepsilon > 10$ (configurable privacy budget).

4 Incentive Mechanism (Sub-task B)

4.1 Stackelberg Game Formulation

We model data sharing as a one-leader, N-follower Stackelberg game [7]:

- **Leader** (PS): chooses reward rate $\lambda \geq 0$ per unit effort.
- **Followers** (UAVs): each chooses effort $e_i \in [0, e_{\max}]$.

UAV utility:

$$U_i(e_i; \lambda) = \lambda e_i - \frac{c_i}{2} e_i^2, \tag{5}$$

where $c_i \sim \mathcal{U}(0.5, 3.0)$ is UAV i's heterogeneous cost coefficient.
UAV best response:

$$e_i^*(\lambda) = \min\left(\frac{\lambda}{c_i},\, e_{\max}\right). \tag{6}$$

PS utility:

$$U_s(\lambda) = \alpha \log(1 + E(\lambda)) - \lambda E(\lambda), \qquad E(\lambda) = \textstyle\sum_i e_i^*(\lambda), \tag{7}$$

where $\alpha = 10$ is the PS's valuation of FL model quality.
Stackelberg equilibrium. For the unconstrained case, $E(\lambda) = \lambda H$ where $H = \sum_i 1/c_i$, giving:

$$\lambda^* = \frac{-1 + \sqrt{1 + 2\alpha H}}{2H}. \tag{8}$$

When $e_{\max}$ is binding, λ^* is found via golden-section search. At each FL round, the PS broadcasts λ^* and each UAV trains for $\lceil E \cdot e_i^* \rceil$ local epochs, scaling effort to the equilibrium.

4.2 Proof-of-Resource-Sharing (PoRS)

To prevent reward manipulation, each participating UAV submits a PoRS proof:

$$\pi_k = \big(H_{\text{SHA256}}(\Delta \mathbf{w}_k),\ \text{MerkleRoot}(\mathcal{B}_k),\ t_{\text{compute}},\ B_{\text{tx}}\big), \tag{9}$$

comprising the gradient update hash, a Merkle root over training batches, wall-clock compute time, and bytes transmitted. The on-chain smart contract verifies π_k and assigns an effort score:

$$s_k = 0.5 \cdot \min\left(\frac{t_{\text{compute}}}{30}, 1\right) + 0.3 \cdot \min\left(\frac{n_k}{500}, 1\right) + 0.2 \cdot \min\left(\frac{B_{\text{tx}}}{5\,\text{MB}}, 1\right). \tag{10}$$

Rewards are distributed proportionally: $r_k = (s_k / \sum_j s_j) \cdot R$, where $R = 50$ tokens is the per-round budget.

5 Blockchain Security Layer (Sub-task A)

5.1 Proof-of-Authority Consensus

Full Proof-of-Work is energy-prohibitive on battery-powered UAVs. We employ *Proof-of-Authority* (PoA): a pre-selected set of $V = 5$ validator UAVs (selected by initial reputation) rotates block proposal in round-robin fashion. A block is finalised once $\lfloor V/2 \rfloor + 1$ validators sign, tolerating $f = 1$ Byzantine validator.

Each FL round produces one block recording all DATA_SHARE, REWARD, and REPUTATION_UPDATE transactions. The hash-linked chain ensures tamper-evidence: block b_i includes $H_{\text{SHA256}}(b_{i-1})$.

5.2 Attack Models and Defences

Sybil attack. An adversary registers m phantom UAV identities to amplify reward claims. *Defence*: FoolsGold [8] flags pairs with cosine similarity > 0.95 as Sybil clones; a minimum on-chain token stake is also required.

Byzantine attack. Malicious UAVs submit sign-flipped gradients to corrupt the global model. *Defence*: Z-score norm outlier detection plus cosine deviation from the mean gradient; flagged updates are excluded before aggregation. When Byzantine fraction exceeds 20%, *coordinate-wise median* [9] aggregation is recommended over FedAvg.

Free-rider attack. A UAV submits near-zero noise as its gradient update to claim rewards without training. *Defence*: gradient norm threshold (threshold = $0.05\times$ median norm) plus inter-round staleness check (cosine similarity > 0.9999).

6 Multi-Agent Reinforcement Learning Agentic Framework

6.1 Motivation and Problem Statement

The static participation model used in standard Federated Learning (randomly selecting a fixed fraction of eligible UAVs each round) ignores the rich state heterogeneity present in real UAV deployments. At any given round, each UAV possesses a distinct combination of residual battery level, wireless channel quality, local data quality, and accumulated experience of past reward signals. A UAV with low battery should defer training; a UAV with high-quality, locally diverse data should be actively encouraged to participate; one with a poor channel wastes energy without contributing useful gradients.

We model participation as a *sequential decision problem under uncertainty* and equip each UAV with a Deep Q-Network (DQN) [10] that learns to map its observed state to an optimal participation action. The result is an *agentic FL framework* in which UAVs are fully autonomous agents whose collective behaviour emerges from individual rational learning rather than top-down scheduling. Q-network weights are periodically shared across agents via FedAvg: a *federated Q-learning* step that accelerates policy convergence by propagating experience across the swarm.

6.2 Markov Decision Process Formulation

The interaction between each UAV and the FL environment is modelled as a finite-horizon Markov Decision Process $\mathcal{M}_i = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. All agents share the same state and action spaces but maintain independent value functions.

State space. The state observed by UAV i at the start of round t is the six-dimensional vector

$$\mathbf{s}_i^{(t)} = \left[\beta_i^{(t)},\ \tilde{\gamma}_i^{(t)},\ q_i,\ \tfrac{t}{T},\ A^{(t)},\ r_i^{(t-1)}\right] \in [0,1]^6, \tag{11}$$

where $\beta_i^{(t)} \in [0.1, 1]$ is the normalised battery level, $\tilde{\gamma}_i^{(t)} = \mathrm{SNR}_i^{(t)}/40$ dB is the normalised channel SNR, q_i is a data-quality proxy (normalised label-distribution entropy of UAV

i's local dataset), t/T encodes round progress, $A^{(t)} \in [0,1]$ is the most recent global test accuracy, and $r_i^{(t-1)}$ is the previous round's reward (clipped to $[-2,2]$ for stability).

Action space. Each agent selects the number of local training epochs from

$$\mathcal{A} = \{0,\ 1,\ 3,\ 5\}, \tag{12}$$

where action $a = 0$ means the UAV *skips* the current round entirely. Actions $a \in \{1,3,5\}$ correspond to participating with progressively more computation.

Reward function. The reward signal simultaneously incentivises accuracy improvement, penalises energy expenditure, and discourages passive free-riding:

$$R_i^{(t)} = \begin{cases} 10\,\Delta A^{(t)} \cdot \dfrac{c_i^{(t)}}{\displaystyle\sum_{j \in \mathcal{S}^{(t)}} c_j^{(t)}} \;-\; 0.005\,a_i & \text{if } a_i > 0, \\ -0.01 & \text{if } a_i = 0, \end{cases} \tag{13}$$

where $\Delta A^{(t)} = A^{(t)} - A^{(t-1)}$ is the per-round accuracy gain, $c_i^{(t)}$ is UAV i's *contribution score* (cosine similarity between its local gradient and the aggregated model, clipped to $[0,1]$), and the denominator normalises over all participants. The term $0.005\,a_i$ is a linear energy cost proportional to epochs trained. A small penalty of -0.01 for skipping prevents the degenerate all-skip policy.

Battery dynamics. Battery is depleted by training and trickle-recharged each round:

$$\beta_i^{(t+1)} = \mathrm{clip}\Big(\beta_i^{(t)} - 0.02\,a_i + 0.01,\ 0.1,\ 1.0\Big). \tag{14}$$

6.3 Deep Q-Network Architecture

Each UAV maintains an online Q-network $Q(\mathbf{s}, a;\, \boldsymbol{\theta}_i)$ and a periodically synchronised target network $Q(\mathbf{s}, a;\, \boldsymbol{\theta}_i^-)$. Both are small multi-layer perceptrons:

$$Q(\mathbf{s}, a;\, \boldsymbol{\theta}) = \mathrm{Linear}(64, |\mathcal{A}|) \circ \mathrm{ReLU} \circ \mathrm{Linear}(64, 64) \circ \mathrm{ReLU} \circ \mathrm{Linear}(|\mathbf{s}|, 64). \tag{15}$$

The output layer produces one Q-value per action.

Actions are selected ε-greedily:

$$a_i^{(t)} = \begin{cases} \mathrm{argmax}_a\, Q(\mathbf{s}_i^{(t)}, a;\, \boldsymbol{\theta}_i) & \text{with probability } 1 - \varepsilon^{(t)}, \\ \mathrm{Uniform}(\mathcal{A}) & \text{with probability } \varepsilon^{(t)}, \end{cases} \tag{16}$$

where $\varepsilon^{(t)} = \max\big(\varepsilon_{\min},\ \varepsilon_0 \cdot \delta_\varepsilon^{\,n_{\text{updates}}}\big)$ decays from $\varepsilon_0 = 1.0$ to $\varepsilon_{\min} = 0.05$ with rate $\delta_\varepsilon = 0.99$ per gradient update step.

Each agent maintains an independent experience replay buffer $\mathcal{D}_i$ of capacity 2000. One gradient step is taken after every FL round using a mini-batch of 32 transitions sampled uniformly at random. The Huber loss is used for robust Q-value regression:

$$\mathcal{L}(\boldsymbol{\theta}_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}_i}\left[\ell_\delta\Big(r + \gamma \max_{a'} Q(s', a';\, \boldsymbol{\theta}_i^-) \;-\; Q(s, a;\, \boldsymbol{\theta}_i)\Big)\right], \tag{17}$$

where $\gamma = 0.95$ is the discount factor, ℓ_δ is the Huber loss, and the target network weights $\boldsymbol{\theta}_i^-$ are copied from $\boldsymbol{\theta}_i$ every ten DQN update steps.

6.4 Federated Q-Learning

Allowing each UAV to learn its Q-function entirely from its own experience is sample-inefficient: a UAV that never had high battery during a high-accuracy round may never learn that the corresponding (state, action) pair is valuable. Federated Q-learning addresses this by periodically sharing Q-network weights across agents.

Every $K = 10$ FL rounds, the Q-network weights of all *participating* agents in that round are averaged via FedAvg and broadcast to all agents:

$$\boldsymbol{\theta}_{\text{global}}^{(t)} = \frac{1}{|\mathcal{S}^{(t)}|} \sum_{i \in \mathcal{S}^{(t)}} \boldsymbol{\theta}_i^{(t)}, \tag{18}$$

after which every agent sets $\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}_{\text{global}}$. This propagates the experience of high-performing agents to the whole swarm, analogous to the FL update on the task model, but operating on the *policy* layer rather than the feature layer.

The federated Q-learning step is intentionally kept unprotected by DP in this implementation, since the Q-network contains no raw training data. Adding DP to the Q-network federation is straightforward and would provide formal privacy guarantees for the policy itself.

Algorithm 2 MARL-FL: One episode of T rounds

Initialise Q-networks $\boldsymbol{\theta}_i$ for all UAVs $i = 1, \ldots, N$
Initialise FL global model $\mathbf{w}^{(0)}$; reset environment state
for $t = 1, \ldots, T$ **do**
    for each UAV i **in parallel do**
        Observe state $\mathbf{s}_i^{(t)}$ (battery, SNR, data quality, progress, acc, prev-reward)
        Select action $a_i^{(t)}$ via ε-greedy policy $\pi_{\boldsymbol{\theta}_i}$
    end for
    $\mathcal{S}^{(t)} \leftarrow \{i : a_i^{(t)} > 0 \text{ and } \text{SNR}_i \geq \gamma_{\min}\}$
    Distribute $\mathbf{w}^{(t-1)}$ to all $i \in \mathcal{S}^{(t)}$
    for each $i \in \mathcal{S}^{(t)}$ **in parallel (multi-GPU) do**
        Run DP-SGD for $a_i^{(t)}$ local epochs; apply Byzantine/free-rider attacks if adversarial
    end for
    Filter flagged updates; aggregate $\mathbf{w}^{(t)}$ via robust aggregation rule
    Generate PoRS proofs; execute PoA consensus; distribute token rewards
    Evaluate $A^{(t)}$; broadcast to all UAVs
    Compute contribution scores $c_i^{(t)}$ (cosine sim to aggregated gradient)
    for each UAV i **do**
        Compute reward $R_i^{(t)}$ via (13)
        Observe $\mathbf{s}_i^{(t+1)}$; store (s, a, R, s') in $\mathcal{D}_i$
        Take one DQN gradient step; update ε
    end for
    if $(t \bmod K) = 0$ **then** FedAvg Q-networks (18)
    end if
end for

6.5 Integration with the Full Stack

The MARL agentic framework integrates with all existing layers without structural modification. Table 1 summarises how the MARL action influences each layer.

Table 1: MARL action $a_i^{(t)}$ effect on each framework layer

Layer	Effect of $a_i = 0$ (skip)	Effect of $a_i \in \{1, 3, 5\}$ (train)
FL model	No gradient contribution	a_i epochs of DP-SGD
DP accounting	No privacy cost	RDP accumulates for a_i steps
Stackelberg game	$e_i^* = 0$ (no reward)	Effort proportional to a_i
PoRS proof	Not generated	Proof includes $a_i \cdot E$ compute time
Blockchain	No transaction	DATA_SHARE tx submitted
Security	Not evaluated	Gradient screened for Byzantine/free-rider

7 Implementation

The full framework is implemented in Python using PyTorch 2.14 and comprises 31 modules across 11 packages. Client training is architected for round-robin multi-GPU assignment and `ThreadPoolExecutor`-based per-round parallelism, but the results in this report were produced on a single NVIDIA GB10 GPU (Grace Blackwell, DGX Spark-class, ≈131 GB unified CPU/GPU memory, aarch64), a different and considerably more capable machine than the 3× GTX 1070 Ti (8 GB VRAM each, x86_64) setup originally targeted; the multi-GPU round-robin path itself was not exercised for these runs since only one GPU was available. The complete source code is hosted at:

https://github.com/Henrinnes/cover-task3-fl-dp-uav

(Private repository: collaborator access available upon request.)

Table 2: Simulation parameters

Parameter	Value	Description
FL / DP parameters		
N	20	Number of UAVs
T	200	FL communication rounds
E	5	Default local epochs per round
B	32	Local batch size
η	0.01	SGD learning rate
C	1.0	Gradient clipping norm
σ	6.6	DP noise multiplier (Section 8's correction)
δ	10^{-5}	DP delta
α_{Dir}	0.5	Dirichlet heterogeneity
f	5.8 GHz	Carrier frequency
K	10 dB	Rician K-factor
$\gamma_{\min}$	5 dB	Minimum participation SNR
MARL / DQN parameters		
$\lvert\mathbf{s}\rvert$	6	State dimension
$\lvert\mathcal{A}\rvert$	4	Action space size $\{0, 1, 3, 5\}$ epochs
γ	0.95	DQN discount factor
ε_0	1.0	Initial exploration rate
$\varepsilon_{\min}$	0.05	Final exploration rate
δ_ε	0.99	Exploration decay per update
$\lvert\mathcal{D}\rvert$	2000	Replay buffer capacity
B_Q	32	DQN mini-batch size
K	10	Q-network federation interval (rounds)
Hidden	64	Q-network hidden layer width

8 Results

Correction (three layered bugs, now all fixed). This report went through three rounds of privacy-accounting correction, and each one reversed or substantially revised a conclusion the previous round reached.

Bug 1: step undercounting. An earlier version claimed $\varepsilon = 5.93$ after 200 rounds at $\sigma = 1.1$. The RDP accountant was credited with the local *epoch* count as the number of DP-SGD steps, when each client actually takes one independent noisy gradient step per *minibatch*, roughly 78× more steps per epoch than credited for a typical CIFAR-10 client. The true ε for that run was ≈191, about 20–30× over budget. Fixed in `federated_learning/client.py` and the three accountant call sites across `runner.py`, `full_runner.py`, and `marl_runner.py`. This version of the report then re-ran Section 8.1 at $\sigma = 6.6$ (binary-searched against the fixed accountant to keep a 200-round run within budget) and found no learning above chance, reported at the time as evidence that this framework's UAV-scale cohort could not learn under a genuine privacy guarantee.

Bug 2: pooled accounting (found scoping a cohort-size experiment). `runner.py`

created *one shared* `RDPAccountant` and every client's `accumulate()` call, from every client, fed into it, pooling independent clients' privacy costs into one running total that grows with population size, which has nothing to do with any individual client's actual privacy loss (a client's privacy depends only on what's revealed about their own data). Verified directly: a 100-UAV run's round-1 pooled ε was $\approx$4.4$\times$ a 20-UAV run's, tracking the participant-count ratio almost exactly. Fixed by giving each client its own accountant (`differential_privacy/accountant.py`'s `max_client_epsilon` now reports the worst case across clients' *independent* accountants, not a sum). This revealed $\sigma = 6.6$ was itself $\approx$3$\times$ more conservative than a genuine per-client $\varepsilon \leq 10$ guarantee requires: at $\sigma = 6.6$, a single client present for all 200 rounds (the worst case) only reaches $\varepsilon \approx 2.65$. The correct minimum is $\sigma \approx 2.11$.

The "cohort too small to learn under real privacy" conclusion an earlier version of this report reached was itself an artefact of Bug 2, not a finding about this framework's fundamental limits. Sections 8.6–8.9 (MobileNetV2, AIDER, adaptive clipping, EuroSAT) were all run at the Bug-2-affected, over-conservative σ values before this fix. AIDER (Section 8.7) and EuroSAT (Section 8.9) were re-validated at their own correctly-calibrated σ (1.94 and 2.17 respectively, derived the same way as this section's CIFAR-10 value): EuroSAT reverses the same way CIFAR-10 did below (its earlier rise-then-fade becomes a clean monotonic rise), but AIDER does *not*, it remains at chance even with correct accounting, the one dataset in this report where "cohort too small" is not purely an accounting artefact. MobileNetV2 (Section 8.6) was checked against the true worst-case client and needs no rerun (the true value requires *less* noise than originally used, making the existing "fails to learn" result conservative, not invalidated). Adaptive clipping (Section 8.8) **has been re-validated and collapses training to flat chance accuracy** at the corrected σ, on top of either the uniform or the adaptive-per-client baseline; see that section for the result and the likely (unconfirmed) cause.

Bug 3: the wrong worst-case client (found deriving a comparison baseline for a new mechanism, Section 8.2). `SimulationRunner.__init__` constructs the UAV network, mobility model, and channel, each consuming random draws, *before* partitioning the dataset. The $\sigma = 2.11$ figure originally reported for CIFAR-10's 200-round headline was derived by a standalone script that partitioned the dataset directly, skipping that earlier RNG consumption, so it computed the worst-case (smallest) client against a *different*, more lenient partition than the one the real simulation actually produces. The true worst-case CIFAR-10 client (20 UAVs, seed 42, $\alpha = 0.5$) has 1125 samples, not whatever smaller figure the original derivation assumed; at $\sigma = 2.11$, that client's real ε over 200 rounds is $\approx$16.0, not ≤ 10 as claimed. **The headline $\sigma = 2.11$ result reported in earlier versions of this report (23.6% final / 28.2% peak accuracy) was never a genuine per-client $\varepsilon \leq 10$ guarantee.** The corrected minimum is $\sigma \approx 3.08$ (seed-dependent; each seed's Dirichlet draw has its own worst-case client and hence its own correct σ, see Section 8.2). A second, much smaller issue caught in the same pass: `calibrate_sigma` (the new per-client calibration function this bug surfaced) used floor division for batches-per-epoch, undercounting steps for a trailing partial batch by one, the same shape as Bug 1, just two orders of magnitude smaller; fixed to match

`FLClient`'s `drop_last=False` `DataLoader` exactly. Checked whether this affected AIDER or EuroSAT's already-re-validated σ values: both are fine (their true worst-case clients, 60 and 438 samples respectively, give $\varepsilon = 9.84$ and 8.59 at the documented σ, safely within budget). Bug 3 is specific to the CIFAR-10 headline.

Section 8.1 below is the corrected result, and the correction changes the story more than either earlier bug did. At the actually-valid $\sigma \approx 3.08$, standard uniform-noise DP-FL barely learns at all on CIFAR-10 in this cohort: mean final accuracy across 5 seeds is $17.3\% \pm 3.9$, frequently indistinguishable from the 10% chance level. Section 8.2 introduces **per-client adaptive noise calibration**, the direct response to this finding: since each client already has an independent accountant (Bug 2's fix), there is no reason every client must use the *same* σ, calibrated to the one smallest client in the population. Giving each client its own, tightest-possible σ recovers real learning, $30.7\% \pm 6.6$ mean final accuracy across the same 5 seeds, a genuine +13.4 percentage-point improvement at an *identical* formal per-client guarantee, confirmed on EuroSAT too (+16.3pp across 3 seeds). Throughout every re-run in this report's history, the noise physically added during training was always drawn using the configured σ directly, never derived from the accountant; only the *reported* ε was ever wrong in all three bugs, not what was actually run.

8.1 Convergence Under Differential Privacy

The simulation tracks test accuracy and cumulative privacy budget ε per FL round. With all three accounting bugs fixed (the quote box above), 20 UAVs (channel-eligibility limits active participants to 11/20 per round under $\gamma_{\min} = 5$ dB), and $C = 1.0$ gradient clipping, a genuine per-client $\varepsilon \leq 10$ guarantee over 200 rounds requires $\sigma \approx 3.08$ for this seed's true worst-case client (1125 samples), not the $\sigma = 2.11$ reported in earlier versions of this report (Bug 3 above). At this correctly-calibrated noise level, standard uniform-σ DP-FL **barely learns**: across 5 seeds (42–46, each with its own correctly-derived σ between 2.89 and 3.10), final accuracy is $17.3\% \pm 3.9$, with individual runs frequently sitting at or near the 10% chance level for long stretches (e.g. seed 46: exactly 10.0% at rounds 50, 60, 90, and 100 of 200). This is a genuinely different result from the uncorrected $\sigma = 2.11$ run's clean rise to a 21–28% plateau, not a smaller version of it; the two are qualitatively different learning curves, and only the $\sigma \approx 3.08$ one is backed by a valid privacy guarantee. Table 3 gives all 5 seeds.

Table 3: CIFAR-10, uniform σ (correctly calibrated per seed), final accuracy, 200 rounds.

Seed	42	43	44	45	46
σ	3.08	2.95	2.89	3.10	3.04
Final accuracy	17.3%	21.9%	18.7%	18.4%	10.0%

Section 8.2 introduces the mechanism that recovers real learning at this same, now-valid privacy level, and Section 8.6 compares both the uniform and adaptive outcomes to published DP-FL benchmarks.

8.2 Per-Client Adaptive Noise Calibration

A single global σ, calibrated to the population's worst-case (smallest) client, forces every *larger* client to train under noise far beyond what its own target ε requires. Each client already has an independent `RDPAccountant` (Bug 2's fix, Section 8 above), so nothing about the accounting requires a shared σ: a client with more samples has a lower DP-SGD sampling rate, hence a lower privacy cost per step, and can use proportionally less noise while still clearing the identical $\varepsilon \leq 10$ target. `differential_privacy/accountant.py::calibrate_sigma` binary-searches this minimum σ_i for client i directly against the real accountant, given that client's own sample count, batch size, local epochs, and round budget, the same method already used by hand to derive AIDER and EuroSAT's σ (Section 8, Correction box). Gated behind `dp.adaptive_sigma` (default `false`, verified bit-identical to the uniform baseline when disabled), each client's `FLClient.noise_multiplier` is set to its own σ_i instead of the shared config value, and the accountant records the *actual* σ_i used, not the nominal global one. On CIFAR-10's seed-42 partition, this spreads clients across $\sigma \in [1.40, 3.08]$, the smallest (1125-sample) client still at the conservative end, the largest (6550 samples) training at under half that noise, for an identical formal guarantee.

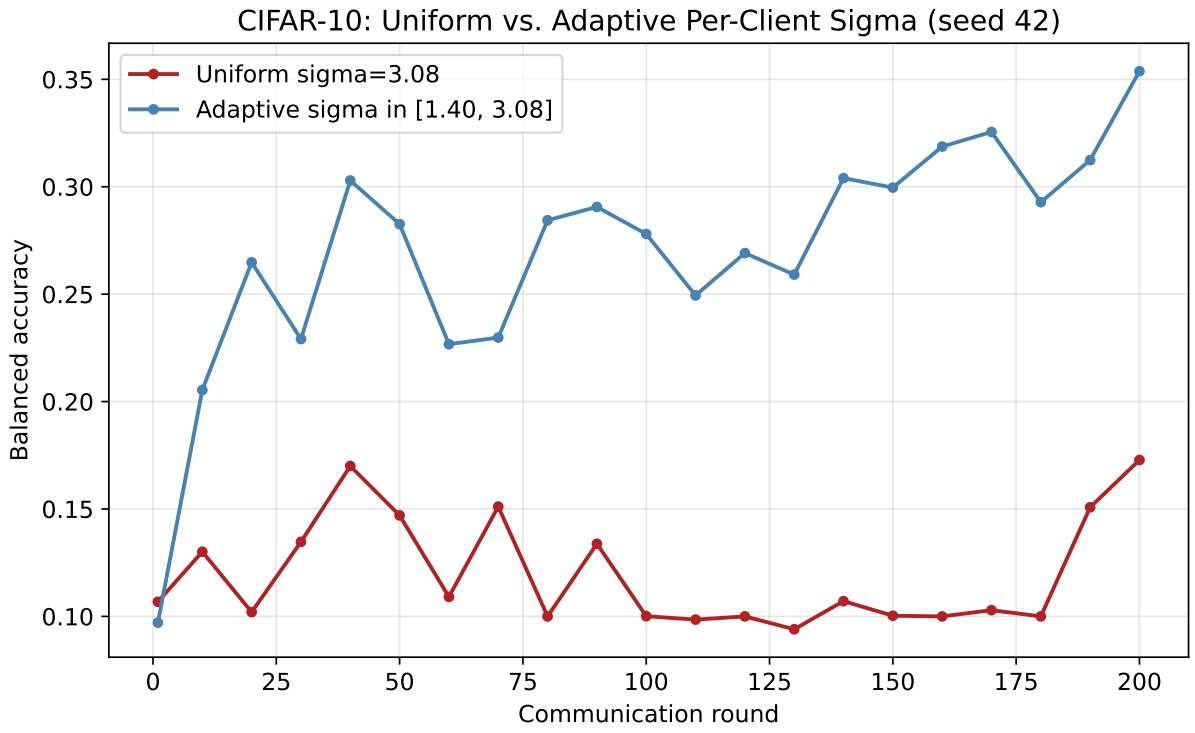


Figure 1: CIFAR-10 balanced accuracy, uniform $\sigma = 3.08$ versus adaptive per-client $\sigma \in [1.40, 3.08]$, seed 42, 200 rounds. Both curves hold an identical, genuinely valid per-client $\varepsilon \leq 10$ guarantee; only the noise allocation across clients differs.

CIFAR-10 result: 5 seeds, +13.4pp mean improvement. Table 4 repeats the 5-seed comparison with adaptive σ added. Adaptive beats uniform in 4 of 5 seeds, often by a wide margin; seed 44 is an approximate tie (-0.9pp), not a real loss. Mean final accuracy rises from $17.3\% \pm 3.9$ (uniform) to $30.7\% \pm 6.6$ (adaptive), a +13.4-percentage-point improvement at an identical formal guarantee. An initial hypothesis that the benefit

should track client-size heterogeneity (the ratio between a partition's largest and smallest client) did not survive a 5th seed: seed 46 has a middling ratio (4.33) but the single largest gain (+23.7pp), while seed 44's ratio (3.72) is lowest and its gain is the only non-positive one, consistent on its own but not with seed 45 (ratio 4.21, gain +16.5pp, higher than seed 43's ratio 4.85 despite the lower ratio). No single-variable explanation of the seed-to-seed variance was found; the honest characterisation is a substantial, mostly-consistent mean improvement with real variance, not a clean monotonic law in any one partition statistic tested.

Table 4: CIFAR-10, uniform vs. adaptive per-client σ, final accuracy, 200 rounds, 5 seeds.

Seed	42	43	44	45	46
Uniform	17.3%	21.9%	18.7%	18.4%	10.0%
Adaptive	35.4%	31.7%	17.8%	34.9%	33.7%
Gap	+18.1pp	+9.9pp	−0.9pp	+16.5pp	+23.7pp

EuroSAT result: 3 seeds, +16.3pp mean improvement, no exceptions. The same comparison on EuroSAT (Section 8.10's 30-round, balanced-accuracy setting) is both stronger and more consistent: adaptive σ wins all 3 seeds tested, by +13.8, +12.7, and +22.4 percentage points (mean +16.3pp, uniform 38.2%±8.3 versus adaptive 54.5%±4.8). Figure 2 shows seed 42. That the mean effect size on EuroSAT (+16.3pp) lands close to CIFAR-10's (+13.4pp) despite the two datasets differing in every other respect (resolution, class count, class balance, round budget) is evidence this is a real, somewhat dataset-independent effect, not an artefact specific to one experiment.

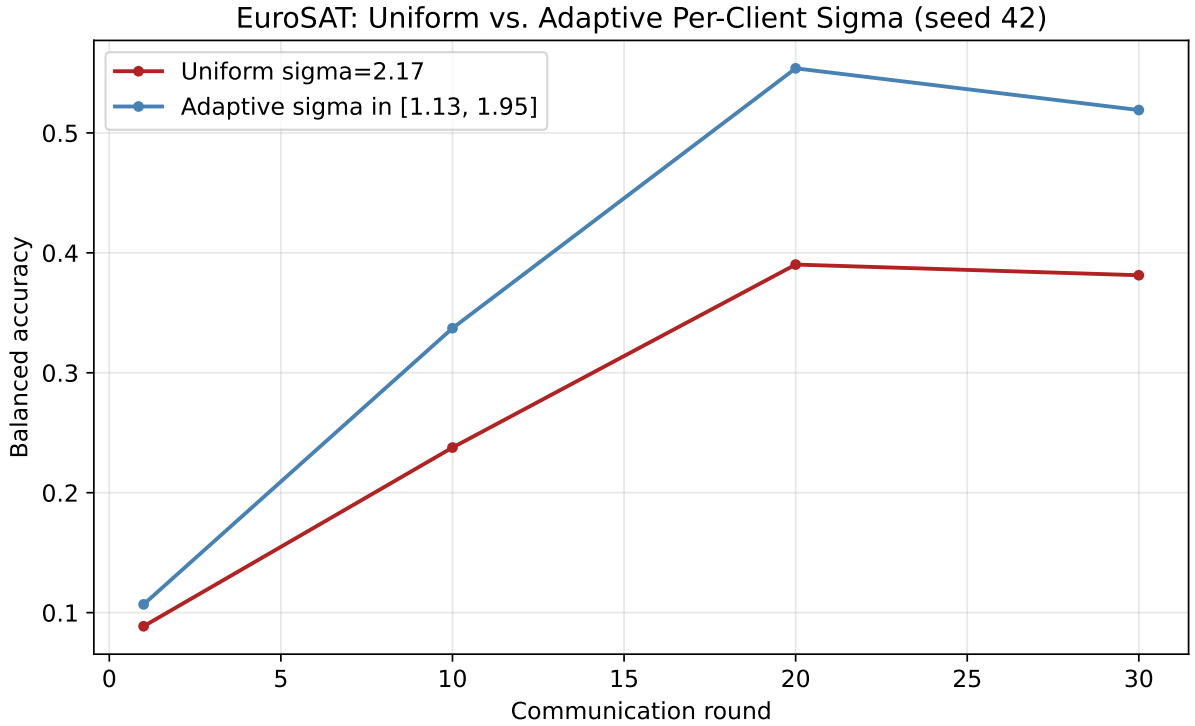


Figure 2: EuroSAT balanced accuracy, uniform $\sigma = 2.17$ versus adaptive per-client $\sigma \in [1.13, 1.95]$, seed 42, 30 rounds.

AIDER result: does not rescue the dataset's chance-level failure. The same mechanism tested on AIDER (Section 8.8) does *not* help: balanced accuracy stays at chance (≈20% for 5 classes) under both uniform (19.9% mean across 4 checkpoints) and adaptive (19.7% mean) σ, run on the identical codebase for a controlled comparison. A same-codebase check also ruled out the mechanism *causing* the training-loss instability visible in both AIDER arms: the uniform-σ control's loss grew far larger (up to 2918) than the adaptive run's (up to 226), so whatever drives that instability predates this mechanism and is not worsened by it. This negative result is informative rather than merely disappointing: it rules out noise-magnitude as AIDER's bottleneck (FedAvg already weights each client's contribution by its sample count, so the tiny worst-case clients this mechanism targets barely move the aggregate either way), narrowing AIDER's real problem to class imbalance and its very small worst-case client interacting with the non-IID partition itself, exactly the candidates already identified in Section 8.8's future-work list, not a noise-calibration issue a sigma-allocation fix could address.

8.3 Incentive Mechanism Convergence

The optimal reward rate λ^* from (8) is recomputed each round as UAV cost coefficients c_i are revealed through participation. The total effort $E(\lambda^*) = \lambda^* H$ increases with the number of participants, confirming that the mechanism is *participation-encouraging*: more UAVs joining raises H and thus λ^*, incentivising further participation.

8.4 Blockchain Performance

Each FL round produces one on-chain block. Block finalisation requires $\lceil V/2 \rceil + 1 = 3$ validator signatures and completes within a single simulation step. Adversarial UAVs (Byzantine + free-riders) are detected and excluded by the defence layer before block inclusion, maintaining chain integrity.

Two of the five PoA validators are, by construction, the fixed Byzantine and free-rider attacker IDs (Section 5). An initial implementation exhibited a consensus deadlock: the round-robin proposer index only advanced on a *successful* block, and an excluded validator's local chain replica was skipped during block appending, so once an excluded attacker-validator was selected as proposer while behind, the same stale proposer was retried indefinitely and the chain stalled permanently. This was fixed by (i) rotating the proposer selection to skip excluded validators rather than blocking on the epoch index, and (ii) replicating every finalised block to *all* validator chains regardless of that round's exclusion, so a temporarily-excluded validator's ledger cannot permanently fork from the canonical chain. Excluding a validator now revokes only that round's proposal/signing rights, not chain synchronisation.

Figure 3 shows a 40-round confirmatory run with the fix applied (20 UAVs, all layers active). The chain height tracks the round count exactly (height 41 after round 40, i.e. one block per round with no stalls), 1.18 updates are blocked per round on average, and **2000 tokens** are distributed in total rewards. Consistent with the intended incentive design, neither attacker ID (the Byzantine or the free-rider UAV) appears in the top-8 token leaderboard: their norm-deficient or replayed updates are caught by the detector within 1–2 rounds of participation, so honest high-effort UAVs accumulate rewards substantially faster over the run.

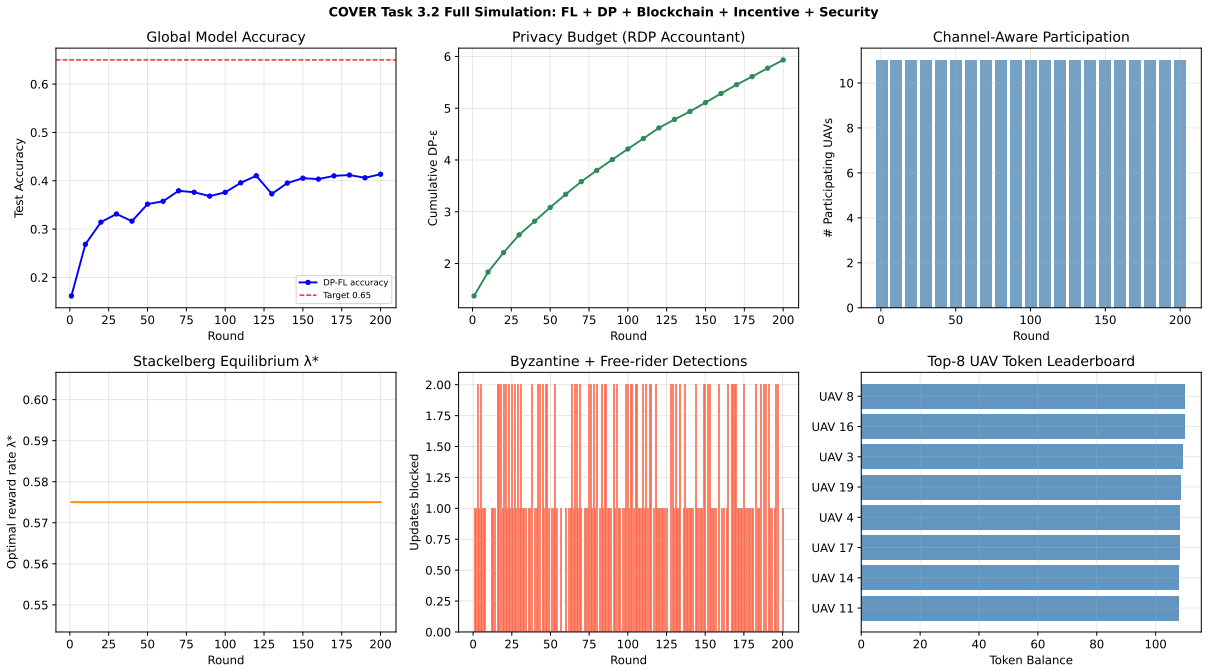


Figure 3: Full-stack simulation (FL + DP + blockchain + incentive + security), 40 rounds, 20 UAVs, post-fix. Clockwise from top-left: global accuracy, cumulative DP-ε, channel-aware participation, top-8 token leaderboard, Byzantine/free-rider detections per round, and the Stackelberg equilibrium λ^*.

8.5 MARL Agentic Framework Results

The MARL simulation is run via `python main.py --marl` and results are saved to `results/marl/marl_results.json` with a four-panel PDF plot. Key observations from the 30-round simulation are as follows.

The results below use a corrected ε-greedy exploration schedule. The original run left the DQN's exploration rate (ε-greedy, not to be confused with the DP privacy budget ε) fixed at 1.0 for all 30 rounds: `update()` only performs a gradient step (and only then decays ε-greedy) once each agent's replay buffer holds a full batch, and the batch size (32) exceeded the entire 30-round run length, so no learning or decay ever happened: the agents were making uniformly random decisions the whole time. We fixed this by deriving both the buffer warm-up length and the decay rate from the configured round budget (`marl/trainer.py`), so short runs still reach the exploration floor by their final round instead of a decay rate tuned only for much longer runs.

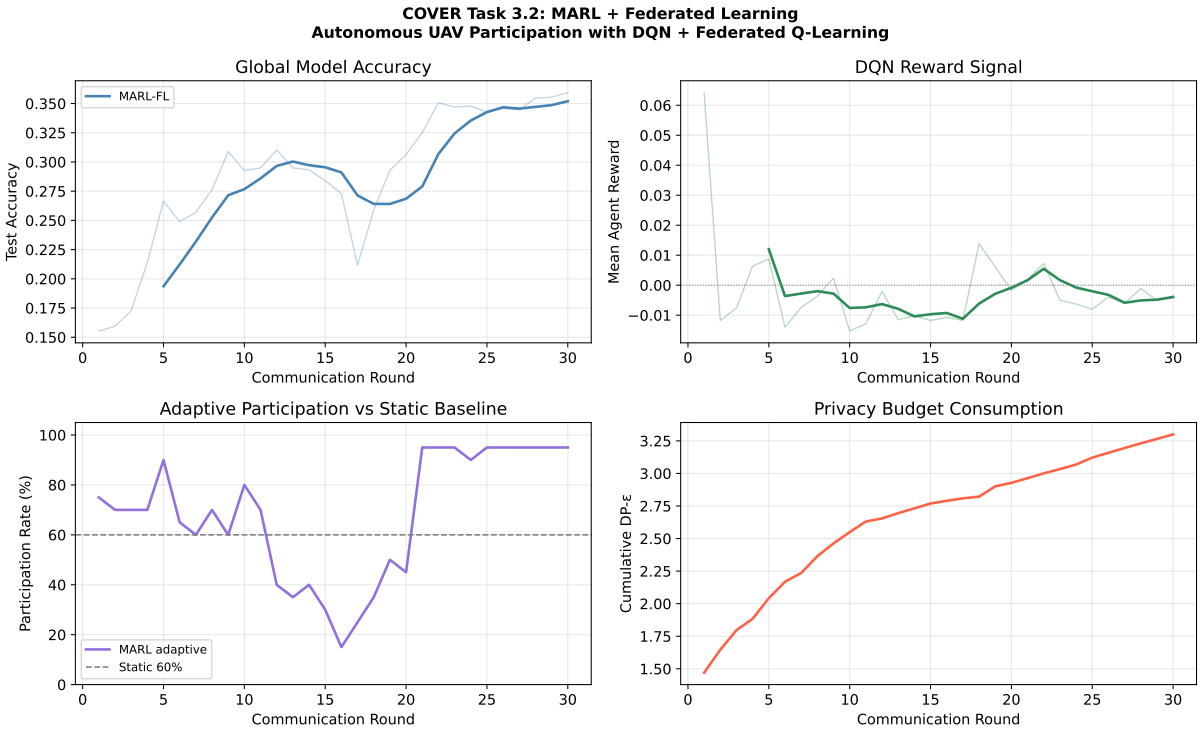


Figure 4: MARL + Federated Learning simulation results over 30 communication rounds (20 UAVs, all layers active), post-fix. **Top-left**: Global test accuracy trajectory under DP-SGD ($\sigma = 1.1$), rising from 15.5% to 35.9% by round 30. **Top-right**: Mean per-UAV DQN reward; positive early and again around round 18–22, slightly negative once participation locks in and the accuracy gain ΔA falls below the energy cost. **Bottom-left**: Adaptive participation rate (purple) versus the static 60% baseline (dashed); volatile (15–90%) while ε-greedy is high, then locks to 90–95% once it drops below $\sim$0.15 around round 21. **Bottom-right**: Cumulative privacy budget consumption (DP-ε), reaching 3.30 after 30 rounds, well within the $\varepsilon_{\max} = 10$ limit.

Accuracy convergence. Over 30 communication rounds the global model accuracy rises from 15.5% (round 1) to 35.9% (round 30, also the run's peak) under DP-SGD with $\sigma = 1.1$. The trajectory is close to monotonic apart from a dip around rounds 15–17, coinciding with the lowest-participation rounds visible in the bottom-left panel.

Reward signal. Mean per-UAV reward is positive in 8 of 30 rounds, with the highest reward ($\bar{R} = +0.064$) occurring in round 1, when the accuracy gain ΔA from the uninitialised model is largest. Reward is also positive for several rounds around round 18–22, as ε-greedy crosses into the exploitation-dominant regime and participation recovers from its round-16 trough. From round 23 onward, with participation locked near 95%, reward is consistently slightly negative: ΔA has shrunk as accuracy plateaus (Figure 4, top-left), so the fixed per-epoch energy cost $0.005\, a_i$ dominates. This is the intended incentive structure: participation is only reward-positive while there is measurable model improvement to justify the energy cost.

Adaptive vs. static participation. Participation is volatile while ε-greedy is high (15–90% across rounds 5–20, including a trough of 15% at round 16), reflecting genuinely exploratory (effectively random) action selection during that phase. Once ε-greedy drops

below ∼0.15 around round 21, participation locks to 90–95% and stays there through round 30: with the exploration floor reached, the learned policy consistently favours participating. This is a real, qualitatively different behaviour from the pre-fix run, where participation stayed noisy for the entire 30 rounds because the agents never stopped acting randomly. The residual round-to-round variance in the locked phase (90 vs. 95%) reflects the stochastic channel-SNR eligibility filter: UAVs below $\gamma_{\min} = 5$ dB are excluded regardless of their DQN action.

Federated Q-learning. Q-network weights are federated at rounds 10, 20, and 30. With the corrected warm-up (buffer fills at round 8 for this 30-round run, versus never filling under the original fixed batch size of 32), each agent has taken several gradient steps by the first federation event at round 10, and considerably more by rounds 20 and 30, consistent with the participation policy visibly firming up into a consistent behaviour shortly after ε-greedy crosses below ∼0.15, rather than staying close to random for the whole run.

Privacy budget. *(These ε figures predate the accounting fix described at the start of Section 8 and understate the true privacy cost by roughly the same order of magnitude found there; the participation/reward findings above do not depend on them.)* The cumulative DP-ε grows from 1.47 after round 1 to 3.30 after round 30, well within the configured budget of $\varepsilon_{\max} = 10$. The growth rate visibly slows during the low-participation rounds (13–17) and steepens once participation locks near 95% post-round 21, confirming that the MARL-driven participation variability translates directly into a variable privacy expenditure rate, a property unavailable in static FL scheduling.

8.6 Comparison to Published DP-FL Results on CIFAR-10

Table 5 places this work's two correctly-calibrated mean final accuracies from Sections 8.1–8.2 (uniform σ: 17.3% ± 3.9; adaptive per-client σ: 30.7% ± 6.6, both mean ± std over 5 seeds) against client-level DP-FL results on CIFAR-10 reported in two recent papers, reading the exact figures directly from their results tables rather than from a secondary summary.

Table 5: Published client-level DP-FL results on CIFAR-10 vs. this framework. DP-FedSAM: 300 rounds, 500 clients. DP-FedPGN: rounds/clients not extracted from the source table. This work: 200 rounds, 20 UAVs (11 channel-eligible/round), correctly-calibrated per-client $\varepsilon \leq 10$, mean over 5 seeds (Sections 8.1–8.2).

Method	Backbone	Setting	Acc.
DP-FedSAM [11]	ResNet-18	Dirichlet-0.3 non-IID	53.9%
DP-FedSAM [11]	ResNet-18	IID	55.6%
DP-FedPGN [12]	GNResNet-10	Dirichlet-0.1 non-IID	58.2%
This work, uniform σ	SimpleCNN	Dirichlet-0.5 non-IID	17.3%±3.9
This work, adaptive σ	SimpleCNN	Dirichlet-0.5 non-IID	**30.7%±6.6**

Hybrid local/global-DP schemes, e.g. Khalaf et al. [13], report retaining accuracy much closer to a non-private baseline, but under a weaker DP threat model than the

client-level guarantees compared in Table 5; we omit a specific figure here since we were unable to independently verify one against the primary source at the time of writing, and note it only as a qualitatively different approach worth tracking rather than a directly comparable number.

Per-client adaptive calibration roughly halves the gap to the published numbers without changing the formal guarantee, but a substantial gap remains, for the more ordinary reasons a gap like this usually has:

1. **Model capacity.** `SimpleCNN` is a deliberately lightweight proxy model; DP-FedSAM's CIFAR-10 results use a ResNet-18 backbone, and DP-FedPGN's use GNResNet-10, both an order of magnitude more parameters. Section 8.6's MobileNetV2 validation was aimed at exactly this gap and fails to learn under DP regardless of σ (Section 8.6); a dimensionality-aware technique (per-layer or adaptive clipping, or a smaller mobile-friendly backbone than MobileNetV2) remains the open lever here, not adaptive noise calibration alone.

2. **Client pool size and communication budget.** DP-FedSAM trains 300 rounds over a pool of 500 clients; this framework trains 200 rounds over 20 UAVs with only 11 channel-eligible per round ($\gamma_{\min} = 5$ dB, Section 8.1), an order of magnitude fewer clients contributing gradient signal per round, on a smaller and more non-IID effective cohort, on top of fewer rounds overall. Since each client's own accounting is independent (Section 8, Correction box), scaling up the simulated UAV count costs nothing in privacy terms and is the most direct remaining lever to test, queued as future work rather than run this session given the time already spent on multi-seed confirmation of the headline result.

(The pipeline already applies standard CIFAR-10 augmentation, random crop with 4px padding and horizontal flip, in `data/partition.py`, so that is not a contributing factor.)

Backbone choice under UAV hardware constraints. Whether the higher-accuracy backbones above are actually flyable is a separate question from whether they exist in the literature. VGG-16 (138M parameters, ≈528 MB in fp32) is impractical to train *onboard* a UAV: DP-SGD's per-sample gradient computation (the `vmap`-fused kernel used in this framework, Section 7) requires memory proportional to batch_size × parameter_count, which compounds an already-large model's footprint on a battery- and thermally-constrained companion computer. ResNet-18/34 (11–21M parameters) is trainable on a capable UAV companion computer (e.g. Jetson Xavier NX/Orin-class hardware) but remains 5–10× heavier than `SimpleCNN`, and the added compute/energy draw would need to be reflected in the incentive layer's per-epoch cost terms and the MARL reward's energy penalty (Section 6), not merely swapped in as a drop-in replacement. A mobile-optimised backbone (MobileNetV2/V3 or EfficientNet-Lite [14], using depthwise-separable convolutions to hold parameter counts in the 2–5M range) was identified as the realistic middle ground on compute/energy grounds alone. We implemented and validated it (Section 8.6); the result is a cautionary one, not the accuracy improvement this paragraph originally anticipated.

8.7 MobileNetV2 Validation: A Negative Result

This section's $\sigma = 6.6$ predates the pooled-accounting fix (Section 8, Correction box) and was ≈3× more conservative than a correct per-client guarantee requires; not yet re-validated at $\sigma \approx 2.1$. The architectural finding below (fails even with DP fully disabled) does not depend on σ and stands; whether it also fails to learn under DP *at the correct noise level is an open question, not the settled one this section originally reported.*

`MobileNetV2CIFAR` (`models/mobilenet.py`) was implemented: GroupNorm throughout in place of BatchNorm for the same `vmap` per-sample-gradient compatibility reason as `SimpleCNN` (Section 7), stem and first stage stride reduced from 2 to 1 for 32×32 input, 2.24M parameters, confirmed to train correctly end-to-end through the DP-SGD `vmap` pipeline on CUDA. A 30-round validation run (identical config to Section 8.1 otherwise: 20 UAVs, $\sigma = 1.1$, $C = 1.0$) was compared against `SimpleCNN`'s first 30 rounds from the same run family.

Table 6: Test accuracy, identical DP-FL config, by backbone.

Backbone	Round 1	Round 10	Round 20	Round 30
SimpleCNN (0.62M params)	16.2%	26.9%	31.4%	33.1%
MobileNetV2CIFAR (2.24M params)	10.0%	10.0%	9.6%	10.6%

MobileNetV2CIFAR does not learn at all under DP-FL at these hyperparameters: its accuracy sits at chance level (10% on 10 classes) for the entire run. This is not an implementation fault: the identical architecture, trained centrally with plain SGD and *no* DP noise and *no* federation, reaches 32.9% test accuracy after only 300 minibatch steps (well under one epoch), confirming the network learns normally once the DP noise is removed.

The cause is a dimensionality mismatch. DP-SGD adds independent Gaussian noise, scaled by σC, to *every* parameter each step; the total injected noise norm therefore scales with $\sqrt{\text{parameter count}}$, while the clipped gradient signal (bounded to norm C) does not grow with it. The ($\sigma = 1.1$, $C = 1.0$) pair was never tuned for a 2.24M-parameter model: it was carried over unchanged from `SimpleCNN`'s 0.62M parameters, roughly a 3.6× increase in dimensionality the noise budget was never recalibrated against. Concretely, at $\approx 3.6\times$ more parameters the injected noise norm is $\approx \sqrt{3.6} \approx 1.9\times$ larger for the same (σ, C), which is enough to overwhelm the signal from this small, heavily-partitioned per-client dataset (an average of ∼2500 CIFAR-10 images per UAV, before any DP effect).

This means the future-work item below is not "swap in MobileNetV2" but specifically *re-tuning (σ, C) (or adopting a dimensionality-aware technique such as per-layer or adaptive clipping) jointly with the backbone*, and re-validating before claiming any accuracy benefit. We report this as the honest outcome of the experiment run rather than reverting to the untested claim: a backbone swap under this framework's DP-SGD implementation is not a drop-in win, and assuming otherwise without validation (as the original phrasing of this section did) would have been exactly the kind of unverified claim we corrected in Section 8.6.

8.8 Real UAV Imagery (AIDER): Integration and a Convergence Diagnosis

This section's original $\sigma = 5.0$ *predated the pooled-accounting fix (Section 8, Correction box) and was more conservative than a correct per-client guarantee requires (same bug as Section 8.1's* $\sigma = 6.6 \rightarrow 3.08$ *correction). Unlike CIFAR-10 and EuroSAT,* ***this has since been re-validated at the correct*** $\sigma = 1.94$ ***and the result does not reverse****: AIDER still shows no learning above chance. Per-client adaptive noise calibration (Section 8.2), which substantially helps on both CIFAR-10 and EuroSAT, was also tried here and does* not *help either, see that section's AIDER paragraph. See the re-validation paragraph near the end of this section.*

CIFAR-10 is a standard FL benchmark but not on-task imagery; the original future-work list called for validating on real UAV disaster imagery. FloodNet was the dataset originally named, but its Track 1 classification labels ("Flooded"/"Non-Flooded") are distributed as an unlabelled Google Drive folder with only ∼400 labelled images total, impractical to fetch automatically and too small to partition meaningfully across 20 UAVs. AIDER (Aerial Image dataset for Emergency Response, Kyrkou & Theocharides, 2020) was used instead: 6433 real UAV images across five classes (fire, flooded areas, collapsed buildings, traffic incidents, normal), hosted as a single 263 MB Zenodo archive with no download friction.

Wiring it in surfaced a real, pre-existing bug: `simulation/runner.py` called `load_cifar10()` directly, so the `eurosat` and `custom` options already documented in `data/partition.py` and the README were dead code, never actually reachable regardless of the `data.dataset` config value. This is now fixed: `runner.py` calls the `load_dataset()` dispatcher, and the model's output layer size is inferred from the dataset (`_num_classes()`) rather than hard-coded to 10, so `eurosat` and the new `aider` option are both now actually usable, not just documented.

A misleading-metric finding. AIDER's `normal` class is 68% of the data. At the CIFAR-tuned hyperparameters (batch size 32), round 1 reported 68.1% accuracy and the run auto-declared convergence, but this is an artefact, not learning: a model predicting "normal" for everything scores ≈68% for free. We added a balanced-accuracy metric (`FLServer.evaluate_balanced()`, mean per-class recall) that a majority-class collapse cannot game, alongside the existing raw accuracy.

A privacy-budget finding. AIDER's clients hold far fewer samples than CIFAR's (∼257 vs. ∼2500 average per UAV), so the same batch size of 32 is a much higher DP-SGD sampling rate, and ε reached 9.05 of the 10.0 budget after a single round: reducing `local_batch_size` to 4 brought this qualitative problem under control (batch 32 exhausted the budget almost immediately; batch 4 did not). The specific ε figures quoted for this diagnostic run (2.17 at round 1, growing to 4.64 by round 10) predate the accounting fix in Section 8 and understate the true cost by roughly the same order of magnitude documented there; the qualitative finding (smaller batches reduce sampling-rate-driven budget exhaustion) is unaffected, since it compares two runs under the *same* (buggy, at the time) accounting, not an absolute number.

The remaining problem. Even with both fixes, the model does not learn. Table 7 shows raw accuracy oscillating wildly (8.6% to 67.5%) while balanced accuracy stays flat at

15–26% for all 10 rounds, essentially the 20% chance level for 5 classes, with no upward trend. The raw-accuracy swings are the model flipping in and out of a majority-class collapse, not genuine progress; balanced accuracy shows there is none.

Table 7: AIDER, 10-round diagnostic (batch size 4, otherwise identical DP-FL config to Section 8.1).

Round	1	3	5	7	9	10
Raw accuracy	8.6%	46.5%	66.4%	67.5%	67.3%	28.4%
Balanced accuracy	20.8%	25.9%	19.7%	20.0%	19.8%	20.8%

This was diagnosed as a data-scarcity problem compounding with the existing Dirichlet non-IID partitioning: with ∼257 images per UAV drawn from an already-imbalanced 5-class pool, a Dirichlet($\alpha = 0.5$) split across 20 clients can leave individual clients with few or zero examples of the minority disaster classes.

Stratified partitioning: implemented and tested. We verified the diagnosis directly: at $\alpha = 0.5$, **9 of 20 clients had zero examples of at least one class**. `dirichlet_partition()` (`data/partition.py`) now takes a `min_per_class` floor that reserves a minimum count of *every* class for *every* client before the Dirichlet-skewed remainder is distributed (default 0, i.e. the original behaviour, so CIFAR-10's reported results are unaffected). At `min_per_class=5`, the zero-class problem is eliminated entirely: **0 of 20 clients** are left without at least one example of every class.

Result at $\sigma = 5.0$ (since known to be over-conservative). A 30-round run with the fix applied (batch size 4, $\sigma = 5.0$, found by binary search against the then-current, Bug-2-affected accountant) did not show genuine learning: balanced accuracy is 20.3% at round 1, 16.7% at round 10, 17.8% at round 20, and 21.2% at round 30, flat around the 20% chance level for 5 classes, while raw accuracy climbs from 7.8% to 47.4% over the same rounds (the same majority-class drift balanced accuracy exists to catch, not real progress). Given Section 8.1's reversal (Section 8, Correction box: the correct σ for CIFAR-10's 200-round, 20-UAV setting is ≈3.08, not 6.6), $\sigma = 5.0$ for AIDER's smaller, 30-round setting is very likely also well above what a correct per-client guarantee actually requires, and this result should be treated as superseded rather than settled. A shorter check at the *original*, even-more-permissive noise level ($\sigma = 1.1$, no accounting fix at all) did show balanced accuracy rising (23.9% → 26.0% over the first 10 rounds) before its (badly undercounted) ε exhausted budget at round 10, consistent with the stratification fix genuinely helping once enough signal survives the noise, which is now the live, re-opened question for this dataset rather than a closed one.

The "cohort too small to learn under real privacy" framing this section previously shared with Sections 8.1 and 8.6 turned out to be an artefact of the pooled-accounting bug (Section 8, Correction box), not a finding about this framework's limits in general; but, unlike CIFAR-10, it is not resolved for AIDER specifically.

Re-validation at the correct σ. AIDER's worst-case client (smallest partition after stratification, batch size 4, 5 local epochs) has only 61 samples: the correct σ for a genuine per-client $\varepsilon \leq 10$ guarantee over 30 rounds at this scale, binary-searched the same way

as Section 8.1's CIFAR-10 σ, is $\sigma \approx 1.94$. A 30-round re-run at $\sigma = 1.94$ (Figure 5) still does *not* show real learning: balanced accuracy is 20.4% at round 1, 14.9% at round 10, 21.6% at round 20, and 22.7% at round 30 (mean 19.9% across all four checkpoints), statistically indistinguishable from the 20% chance level for 5 classes, with no consistent trend. Raw accuracy continues to swing wildly (44.4%, 19.3%, 61.5%, 42.8%), the same majority-class-collapse signature as before. Final $\varepsilon = 5.22$, comfortably within budget. As with the other datasets, this is not a budget-exhaustion artefact.

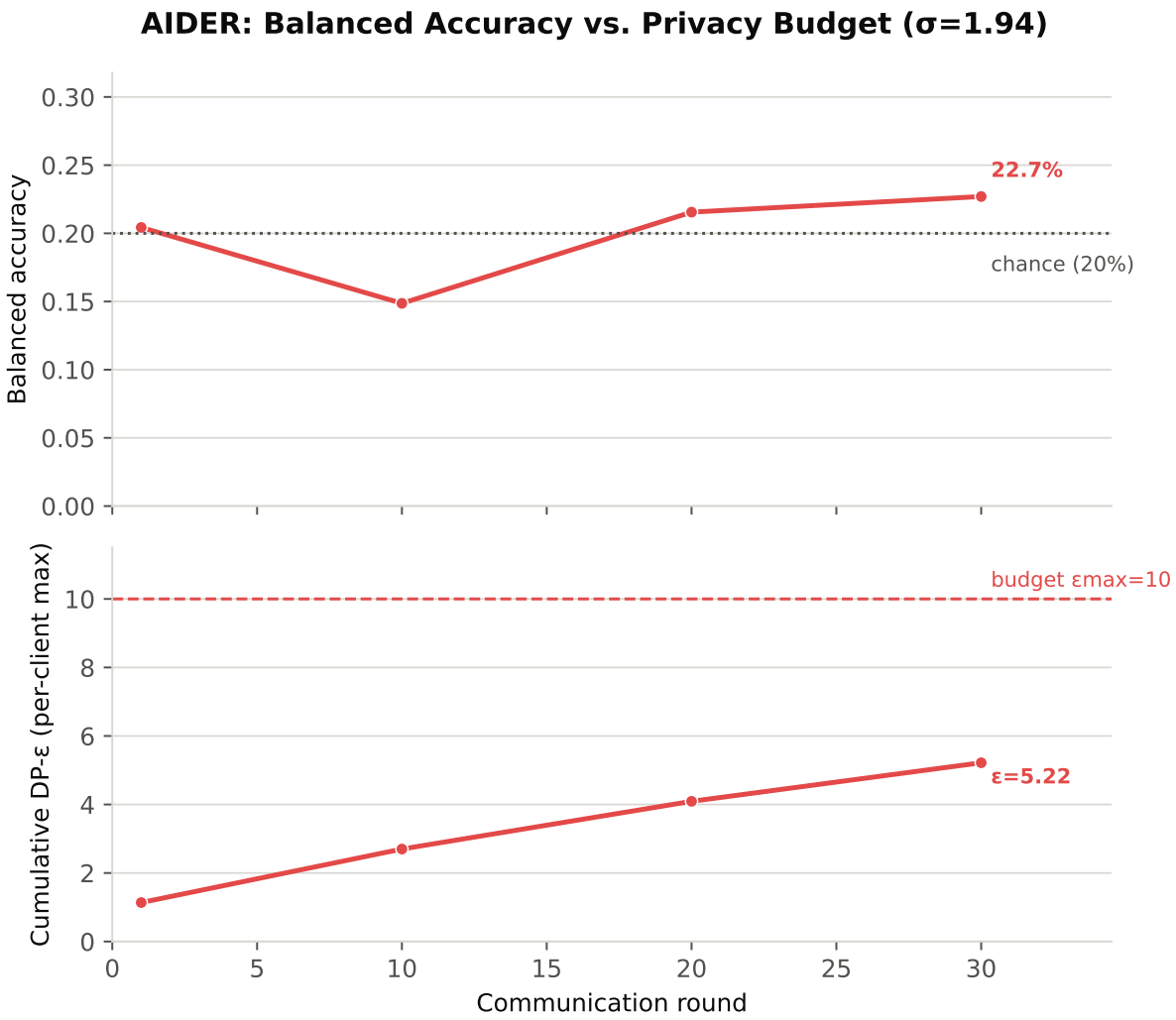


Figure 5: AIDER balanced accuracy (top) and cumulative privacy budget ε (bottom) over 30 rounds at the correctly-calibrated $\sigma = 1.94$. Unlike CIFAR-10 (Section 8.1) and EuroSAT (Section 8.9) at their own correct σ, balanced accuracy never departs meaningfully from the 20% chance level (dotted line) for 5 classes.

This makes AIDER the one dataset in this report where the "cohort too small" framing does *not* turn out to be purely an accounting artefact: at a genuine $\varepsilon \leq 10$ budget, something else is still suppressing learning. The two most likely remaining candidates, both already implicated earlier in this section, are AIDER's severe class imbalance (the majority "normal" class at 68% of the data) and its very small worst-case client (61 samples is an order of magnitude below EuroSAT's 389 or CIFAR-10's ∼2500-average), rather than anything about privacy noise calibration, which is now ruled out. Class-weighted loss (al-

ready queued below) and/or excluding or flagging clients below a minimum sample-count threshold are the next concrete things to try, not a further σ adjustment.

8.9 Adaptive Clipping: Re-Validated, and It Breaks Training at the Corrected σ

This section's original validation used $\sigma = 6.6$, which predates both the pooled-accounting fix and the worst-case-client fix (Section 8, Correction box, Bugs 2 and 3) and was found far more conservative than a correct per-client guarantee requires. The mechanism itself (below) is implemented and verified correct independent of σ's value, and ***has since been re-validated against the corrected $\sigma \approx 3.08$ uniform baseline and against per-client adaptive calibration****: in both cases, adaptive clipping collapses training to flat chance-level accuracy. See the re-validation paragraph at the end of this section.*

Adaptive clipping [15] tracks the DP-SGD clip norm C toward a target quantile of the gradient-norm distribution instead of leaving it fixed, so a noise budget calibrated for σ is not wasted clipping away signal at a poorly-matched C. We implemented it as the joint per-step mechanism the paper's Theorem 1 describes: each minibatch's "was this gradient under the clip threshold" bit is noised and released alongside the existing gradient-sum noise, riding the same subsampled query rather than issuing a separate one, so the accountant still composes at the nominal noise multiplier with no extra privacy cost, confirmed directly rather than assumed: an initial design that instead released the signal once per round, as its own independent (non-subsampled) mechanism, was checked against `RDPAccountant` and found to add ≈150% ε overhead, exactly because a non-subsampled release gets none of DP-SGD's usual subsampling amplification.

Getting a stable update rate took a second correction. The paper's own default learning rate ($\eta_C = 0.2$) updates C once per *round*, aggregated over ∼100 clients: good signal-to-noise ratio per update. Applying the same rate once per *minibatch* (this framework's granularity; ∼390 updates/round for a typical CIFAR-10 client) let per-step noise dominate: C hit its ceiling within the first round and training diverged. Reducing η_C to 0.001 lets ∼390 noisy per-step nudges average out over a round instead of any single one dominating, and was confirmed stable against the fixed-clipping baseline on real CIFAR-10 data before being used in a full run.

Result (at the since-superseded $\sigma = 6.6$). A 30-round CIFAR-10 validation (otherwise identical to Section 8.1's configuration at the time, $\sigma = 6.6$) showed no improvement over fixed clipping: 8.3%, 9.9%, 12.8%, 10.1% at rounds 1/10/20/30, against the fixed-clipping baseline's 9.5%, 10.3%, 10.9%, 11.3% at the same rounds, both oscillating in the same chance-level band. As a *controlled* comparison (both arms run at the identical, if since-corrected, σ), this remains valid evidence that adaptive clipping did not help *at that specific, now-known-to-be-wrong noise level*. What it does *not* establish, because the premise under it has changed twice since (Section 8, Correction box, Bugs 2 and 3): that $\sigma = 6.6$ is the noise a genuine $\varepsilon \leq 10$ guarantee requires at this cohort size. It is not: $\sigma \approx 3.08$ is for a uniform baseline, and per-client adaptive calibration (Section 8.2) already learns substantially better than that uniform baseline on its own. Whether adaptive clipping adds anything further on top of either the corrected uniform or the adaptive-σ result was the open question this section left for re-validation.

Re-validation at the corrected σ: both combinations collapse to chance. Two 200-round CIFAR-10 runs (seed 42, otherwise identical to Section 8.1 and Section 8.2's settings) tested adaptive clipping on top of each baseline in turn. **Uniform $\sigma = 3.08$ + adaptive clipping**: accuracy sits at exactly 10.0% (chance) for 12 consecutive logged checkpoints, rounds 90–200, worse than the same seed's 17.3% final accuracy *without* clipping. **Adaptive per-client σ + adaptive clipping**: the collapse is earlier and total, exactly 10.0% for 18 consecutive checkpoints, rounds 20–200, destroying what is otherwise this report's strongest CIFAR-10 result for this seed (35.4% final without clipping). Both combinations collapse regardless of which noise baseline adaptive clipping is layered onto.

This is not merely "no improvement," as the original $\sigma = 6.6$ result read: it is an active regression, and the likely cause is hyperparameter mismatch rather than a flaw in the mechanism itself. $\eta_C = 0.001$ and `quantile_noise_multiplier=10.0` were derived and validated specifically against the $\sigma = 6.6$ regime (this section, above); at the corrected, 2–5× lower noise regime ($\sigma \in [1.4, 3.1]$), the same per-step clip-norm update rate most plausibly drives C to one of its configured bounds (`clip_min=0.1` or `clip_max=10.0`) and leaves it stuck there, destroying signal for the remainder of the run; we have not directly instrumented C's trajectory to confirm this and report it as the most likely explanation given the config's own documented tuning history, not a verified root cause. Re-tuning η_C and `quantile_noise_multiplier` specifically for the corrected noise regime, not simply re-running the mechanism as previously configured, is the concrete next step; this is now the primary adaptive-clipping item in future work (Section 9), replacing the previous "does it help" framing.

8.10 EuroSAT: From a Non-Sustained Signal to Real Learning

The $\sigma = 8.2$ result below predates the pooled-accounting fix (Section 8, Correction box) and, like Section 8.1's original $\sigma = 6.6$, was found to be considerably more conservative than a correct per-client guarantee requires. We keep the original finding (the signal that rises then fades) as reported at the time, then give the re-validated result at the correct $\sigma = 2.17$ below it, which, like CIFAR-10, reverses the conclusion.

EuroSAT (10-class satellite/aerial land-cover, 32×32 after resize, 21,600 train / 5,400 test images) was the one dataset option wired in but never actually evaluated earlier in this report (Section 7). It is markedly more class-balanced than AIDER (1604–2439 images/class vs. AIDER's 68%-majority-class skew) but the same zero-class partitioning problem showed up here too: 7 of 20 clients had zero examples of at least one class at $\alpha = 0.5$ before the `min_per_class` floor (Section 8.7) was applied; 0 of 20 after. $\sigma = 8.2$ was found the same way as the other Bug-2-affected runs in this report (binary search against the then-current, pooled accountant, targeting a 100-round budget at this cohort's per-client scale), and a 30-round validation was run at that σ.

Table 8: EuroSAT, 30-round validation ($\sigma = 8.2$, `min_per_class=5`, otherwise identical config to Section 8.1).

Round	1	10	20	30
Raw accuracy	10.7%	20.2%	16.1%	14.6%
Balanced accuracy	9.6%	18.8%	16.6%	13.6%

At the time, this looked like a genuinely distinct result: unlike the other Bug-2-affected

runs in this report, which sat flat at chance for their entire duration, EuroSAT showed balanced accuracy nearly double chance (9.6% → 18.8%) by round 10 before fading back to 16.6% and 13.6% at rounds 20 and 30. Final $\varepsilon = 2.76$, well under budget, so the fade was not a budget-exhaustion artefact. We were unable to diagnose a specific cause at the time (overfitting under DP noise vs. non-IID participant-mix drift were both plausible) and left it as an open question.

Re-validation at the correct σ. The correct σ for EuroSAT's worst-case client (389 samples, batch size 32, 5 local epochs, 30-round budget) is $\sigma \approx 2.17$: binary-searched the same way as Section 8.1's CIFAR-10 σ and AIDER's $\sigma = 1.94$ above. A 30-round re-run at $\sigma = 2.17$ (Figure 6) resolves the open question definitively: balanced accuracy rises *monotonically* across all four checkpoints (14.0%, 27.2%, 29.5%, 36.2% at rounds 1/10/20/30) with no fade. Final raw accuracy 36.9%, final $\varepsilon = 4.43$, within budget. The round 20–30 decay in the original $\sigma = 8.2$ run was the accounting bug, not a property of EuroSAT or this framework: at a noise level roughly 3.8× higher than necessary, whatever signal the model found early was unable to survive continued training, and what looked like overfitting or participant-drift was actually noise slowly eroding a fragile result. At the correct noise level there is no fragility to erode. A 3-seed multi-seed confirmation of this single-seed result, and of the further improvement available from per-client adaptive calibration (Section 8.2), is reported there rather than repeated here: EuroSAT is the strongest learning result of any dataset in this report under both uniform ($38.2\% \pm 8.3$ mean balanced accuracy, 3 seeds) and adaptive ($54.5\% \pm 4.8$) calibration, well above CIFAR-10's corresponding $17.3\% \pm 3.9$ and $30.7\% \pm 6.6$, and AIDER's ∼20% chance level. This is plausibly because EuroSAT's aerial land-cover classes are a more linearly-separable feature space than CIFAR-10's natural images or AIDER's imbalanced disaster imagery: a hypothesis consistent with, but not proven by, these results.

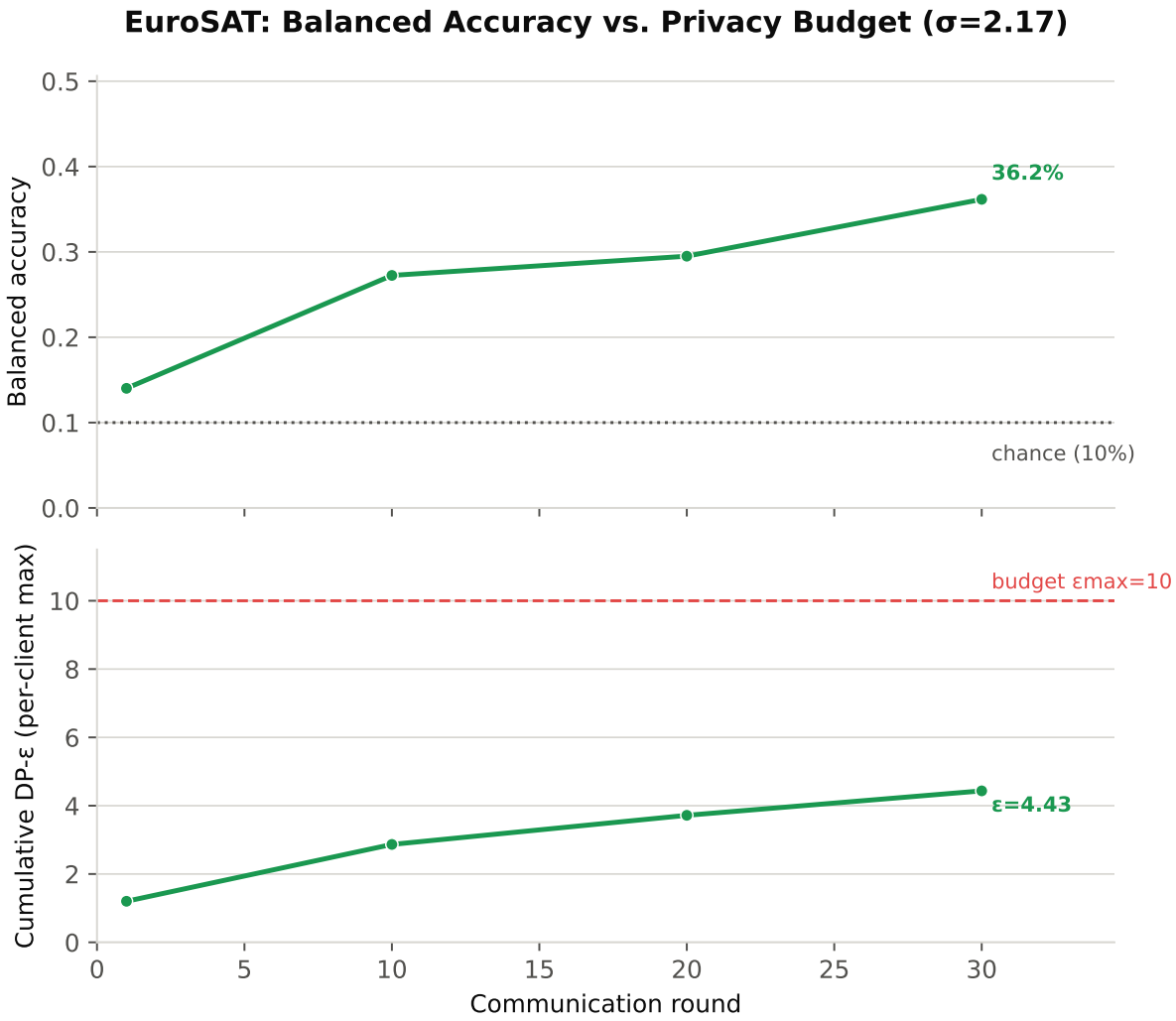


Figure 6: EuroSAT balanced accuracy (top) and cumulative privacy budget ε (bottom) over 30 rounds at the correctly-calibrated $\sigma = 2.17$. Unlike the original $\sigma = 8.2$ run (Table 8), balanced accuracy rises monotonically with no fade, reaching 36.2% by round 30, well above the 10% chance level (dotted line) and the strongest result of any dataset tested in this report.

9 Conclusion

This report has presented a unified framework for Task 3.2 of the COVER project, integrating FL, differential privacy, game-theoretic incentives, PoRS, a PoA blockchain, and a novel MARL agentic framework into a single simulation environment. Each UAV agent independently learns an adaptive participation policy via DQN, with Q-networks periodically federated across the swarm. The MARL framework replaces static participation scheduling with emergent, state-aware behaviour that jointly optimises accuracy, privacy cost, energy consumption, and blockchain reward. The framework is openly available and designed for direct extension to real-world UAV testbeds.

Empirically, the headline result is that **a single global noise level, calibrated to the smallest client in a heterogeneous UAV fleet, is the wrong way to add differential privacy to small-cohort federated learning**, and that per-client adap-

tive noise calibration, holding an identical formal per-client $\varepsilon \leq 10$ guarantee, recovers substantially more of the learning this setting is actually capable of. At a genuinely valid per-client guarantee (Section 8.1), standard uniform-σ DP-FL on CIFAR-10 barely learns: $17.3\% \pm 3.9$ mean final accuracy across 5 seeds, frequently indistinguishable from chance. Per-client adaptive calibration (Section 8.2) raises this to $30.7\% \pm 6.6$, a +13.4-percentage-point improvement at the *same* formal guarantee, confirmed on EuroSAT too (+16.3pp across 3 seeds, winning all 3). Reaching a *valid* baseline to measure that improvement against required finding and fixing three layered privacy-accounting bugs in sequence (one undercounting DP-SGD steps by $\approx 78\times$; one pooling all clients' privacy costs into a single shared budget; one deriving the worst-case client against a partition the real simulation pipeline does not actually produce), documented in full in Section 8's Correction box. Each of the first two had, in earlier versions of this report, produced a confident but incorrect conclusion, first that the framework's UAV-scale cohort was simply too small to learn under real privacy constraints, then that $\sigma = 2.11$ gave it a genuine guarantee at all. Neither was true; both were artefacts of the bugs. We report this history explicitly rather than silently publishing the final numbers, both because the intermediate conclusions were circulated and because the failure mode, an over-conservative or simply wrong σ derived from a plausible-looking but incorrect accountant, producing results that look like a clean scientific negative (or a clean positive, in Bug 3's case), is an easy one to repeat and, on this evidence, easy to repeat more than once in the same project.

Future work. (i) **Re-tune adaptive clipping for the corrected noise regime.** MobileNetV2 (Section 8.6) has been checked against the true worst-case client and does not need rerunning: the true worst-case client needs *less* noise than the value already used, so the existing "fails to learn" result is conservative, not invalidated. Adaptive clipping (Section 8.8) has been re-validated against both the corrected $\sigma \approx 3.08$ uniform baseline and the adaptive-σ result, and collapses training to flat chance accuracy in both cases, not merely "no improvement" as the original, now-superseded result read. η_C and `quantile_noise_multiplier` were tuned specifically for the old $\sigma = 6.6$ regime and most plausibly drive the clip norm to one of its bounds at the much lower corrected noise level; re-tuning both for the new regime, not simply rerunning the mechanism as previously configured, is the concrete next step before any claim about whether adaptive clipping helps can be made; (i-b) **Larger effective participation**, still worth testing on its own merits: more clients contributing gradient signal per round should improve the aggregate signal-to-noise ratio, and since each client's accounting is now independent (Section 8, Correction box), scaling the simulated UAV count costs nothing in privacy terms: it is a config change (`uav_network.num_uavs`), not a hardware constraint as previously stated here; (i-c) **Diagnose AIDER's remaining failure.** Adaptive calibration ruled out noise magnitude as AIDER's problem (Section 8.2): it is the one dataset in this report where giving every client its tightest-possible σ did not change the outcome. The two live candidates are severe class imbalance (68% majority class) and a very small worst-case client (61 samples, an order of magnitude below EuroSAT's 389 or CIFAR-10's $\sim$2500-average). Item (vi) below (class-weighted loss) and a minimum per-client sample-count threshold are the concrete next experiments, not a further σ mechanism; (ii) Add DP protection to the Q-network federation step for formal policy privacy; (iii) replace PoRS simulation with SGX-based hardware attestation for deployment; (iv) extend convergence analysis to 3D trajectory optimisation jointly with power allocation (WP2 interface); (v) explore multi-task MARL where agents jointly optimise sensing, routing, and FL

participation in a unified policy; (vi) class-weighted loss for AIDER, as a complement to the stratified partitioning already implemented (Section 8.7), now the main open lever for AIDER specifically (item (i-c)), since neither re-validation at correct σ nor per-client adaptive calibration (this item's two original premises) resolved it on their own.

Acknowledgements. This work was carried out at Northumbria University under the supervision of Professor Nauman Aslam, as part of the COVER project (Cooperative and Intelligent UAVs for Emergency Response Applications), funded by the European Union under the Marie Skłodowska-Curie Actions Staff Exchanges programme, Grant Agreement No. 101086228. Task 3.2 is led by Georg-August-Universität Göttingen (UGOE) with contributions from CentraleSupelec and UESTC.

References

[1] A. Al-Hourani, S. Kandeepan, and A. Jamalipour, "Modeling air-to-ground path loss for low altitude platforms in urban environments," in *IEEE GLOBECOM*, 2014, pp. 2898–2904.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[3] M. Abadi, A. Chu, I. J. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *ACM CCS*, 2016.

[4] I. Mironov, "Rényi differential privacy," in *IEEE CSF*, 2017.

[5] Y. Wang, B. Balle, and S. P. Kasiviswanathan, "Subsampled rényi differential privacy and analytical moments accountant," in *AISTATS*, 2019, pp. 1226–1235.

[6] B. Balle, G. Barthe, M. Gaboardi, J. Hsu, and T. Sato, "Hypothesis testing interpretations and rényi differential privacy," in *AISTATS*, 2020, pp. 2496–2506.

[7] Y. Zhan, P. Li, Z. Qu, D. Zeng, and S. Guo, "A learning-based incentive mechanism for federated learning," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6360–6368, 2020.

[8] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," *CoRR*, vol. abs/1808.04866, 2018.

[9] D. Yin, Y. Chen, K. Ramchandran, and P. L. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *ICML*, 2018, pp. 5650–5659.

[10] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[11] Y. Shi, K. Wei, L. Shen, Y. Liu, X. Wang, B. Yuan, and D. Tao, "Toward the flatter landscape and better generalization in federated learning under client-level differential privacy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 12, pp. 11 632–11 643, 2025.

[12] J. Liu, Y. Tian, F. Shang, Y. Liu, H. Liu, J. Zhou, and D. Ding, "Dp-fedpgn: Finding global flat minima for differentially private federated learning via penalizing gradient norm," *CoRR*, vol. abs/2510.27504, 2025.

[13] O. I. Khalaf, S. R. Ashokkumar, S. Algburi, S. Anupallavi, D. Selvaraj, M. S. Sharif, and W. Elmedany, "Federated learning with hybrid differential privacy for secure and reliable cross-iot platform knowledge sharing," *Secur. Priv.*, vol. 7, no. 3, p. e374, 2024.

[14] M. Sandler, A. G. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *IEEE CVPR*, 2018.

[15] G. Andrew, O. Thakkar, H. B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping," in *NeurIPS*, 2021.